

SE 4485: Software Engineering Projects

Fall 2026

Project Management Plan

Group Number	Group 9
Project Title	Skills Catalog & Inventory (Team A)
Sponsoring Company	Sabre Corporation
Sponsor(s)	Jeff Buchmiller
Students	1. Tobenna Nwosu 2. Jason Lehman 3. Alice Askerup 4. Russel Costales 5. Kim Chau 6. Jasmine Shrestha

Skills Catalog & Inventory

A. ABSTRACT

This is the Project Management Plan (PMP) for the Skills Catalog & Inventory project sponsored by Sabre Corporation. This project is one of the ten projects for this course SE 4485.001 at the University of Texas at Dallas. This document complies with the Software Engineering Standards set by the IEEE Standard for Software Project Management Plans (IEEE Std 1058-1998). The PMP guides the project through different phases and helps set standards, timelines, tasks, and plans.

B. TABLE OF CONTENTS

A. Abstract.....	3
B. Table of Contents.....	4
C. List of Tables.....	5
1. Introduction.....	5
1.1. Introduction to the document.....	5
1.2. Purpose and Scope of the document.....	5
1.3. Brief overview of the product.....	5
1.4. Description of the structure of the document.....	6
2. Project Organization.....	6
3. Lifecycle Model Used.....	6
4. Risk Analysis.....	6
4.1. Team Risks.....	7
4.2. Technical Risks.....	7
4.3. External/Sponsor Risks.....	8
5. Software and Hardware Resource Requirements.....	9
6. Deliverables and Schedule.....	9
7. Monitoring, Reporting, and Controlling Mechanisms.....	10
8. Professional Standards.....	11
9. Configuration Management.....	11
10. Engineering Standards and Multiple Constraints.....	13
11. Additional References.....	13
Appendix A. Professional Standards Guideline.....	13

C. LIST OF TABLES

1. Team Risks
2. Technical Risks
3. External/Sponsor Risks
4. Deliverables and Schedules
5. Configuration Management

1. Introduction

1.1. Overview

The purpose of this project is to build a cloud-based software tool to help organizations overcome the challenge of tracking employee skills across complex team structures. To make the workforce capabilities clear and connected, the platform links three core areas: the Skills Catalog/Master Data (standardized definitions and hierarchy of all skills), Organization Structure (company's teams, roles, and relationships), and Skills Inventory (mapping between the employees and their skills).

1.2. Purpose and scope of the document

The purpose of this document is to define the functional and structural requirements for developing the cloud-based software. As the semester goes on, we will explore and choose what our tech stack would be to satisfy the requirements. The scope of the document is as follows:

1. Core Data Architecture: Designing the relationship between Master Data, Organization Structure, and Operational Data
2. Stakeholder Groups:
 - a. Workforce Planners - Build & maintain the Skills catalog
 - b. Resource Managers - Enter, view, and update Skills per resource, creating and maintaining the Skills Inventory
 - c. Organizational Leaders - Skills inventory summary views, in order to manage the existing organization and plan future workforce strategies
 - d. Business Operations - Load files of additional Skills into the Skills catalog and of additional skills per resource into the Skills Inventory

1.3. Brief overview of the product

A cloud-based software tool has to be designed and built to create and maintain a Skills Catalog. The tool has to be built with a focus on an exceptional user interface (UI) with the ability to enter, view, and update skills per resource, thereby creating the Skills Inventory. The purpose of this tool is to simplify the process of keeping track of which resources in the company have which skills and create a skills catalog that users can easily view, understand, and update. In the organization, leaders view summaries of skill inventory to manage organizational skills capacity and plan future workplace strategies.

1.4 Description of the structure of the document

This Project Management Plan provides a structured roadmap for the development of the Skills Catalog & Inventory system. The document is organized into the following sections:

1. Introduction: Defines the project's purpose, scope, and product overview.
2. Project Organization: Identifies the team structure, including leadership and member roles.
3. Lifecycle Model Used: Details the Waterfall methodology adopted for the project's development.
4. Risk Analysis: Outlines potential team, technical, and external risks, along with mitigation strategies.
5. Software and Hardware Resource Requirements: Specifies the tools and environment for development.
6. Deliverables and Schedule: Provides a timeline for project phases, from requirements analysis to final reporting.
7. Monitoring, Reporting, and Controlling Mechanisms: Describes how the team will track progress and quality.
8. Professional Standards: Sets expectations for team conduct and deliverables.
9. Evidence of Configuration Management: Links to relevant repository management tools.
10. Engineering Standards and Multiple Constraints: Lists the industry standards guiding the project.
11. Additional References & Appendix: Includes supplementary material and the professional standards guideline.

2. Project Organization

This team of collaborators is working together to create this program for Saber Corporation. Tobenna is the team leader, while Alice, Jason, Kim, Jasmine, and Russel are team members. This is because the project is still in its early stages of development and research; it is best for everyone to work together before branching into different roles.

3. Lifecycle Model

For this project, we will be using Waterfall as our lifecycle model. It is the best for documenting our research and discoveries for this project, as well as allowing us time to code the project properly. Waterfall was designed with a focus on thorough documentation and progression through sequential stages, which closely matches the nature of this class and project.

4. Risk Analysis

Problems may arise from the team, technical, or external factors. These issues may be exacerbated by inadequate planning, as some incidents are unforeseeable. Risks are managed by indicating the likelihood of occurrence and planning strategies to address them. The likelihood of each risk is based on the

constraints of the project, such as but not limited to: development time, use of the Waterfall method, team size, and unfamiliar technologies.

4.1. Team Risks

Risk	Likelihood	Strategies
Miscommunication may create duplicate work or create inconsistent features.	Medium	Using shared task boards like Trello to track task ownership and feature progress.
Varying skill levels or task assignments can create uneven workload distribution.	Medium	Distribute tasks relative to each member's strengths and comfort level.
User authentication and validation may need to be more robust than team members have experience with building.	High	Utilizing a prebuilt authentication solution can reduce implementation risk.
A team member may be unreachable or unresponsive during development and documentation.	Low	Task ownership is visible on a shared task board for them to be reassigned and redistributed.
A member or multiple members of the group decides to drop the class.	Low	Avoid critical tasks from being assigned to a single member, so tasks can be picked up by another member without onboarding delays.

4.2. Technical Risks

Risk	Likelihood	Strategies
Local development environments can be inconsistent, creating issues on some machines.	High	Create setup documentation and shared requirements/dependency files to prevent mismatched versions.
Critical bugs may be revealed late due to the lifecycle process, which may be close to deadlines.	Medium	Test each feature upon completion, and do not wait for integration.
Merge conflicts or overwritten work may extend development time depending on severity.	Medium	Require multiple members of the group to approve a pull request.
Underestimated development complexity of certain features or work.	High	Complex features should be broken down into smaller milestones. This also allows for

		features to be gradually tested.
Performance problems develop as real data is implemented if it exceeds test data expectations	Medium	Test on varying amounts of data and seek guidance from the sponsor on likely amounts that will be implemented.
Cloud infrastructure and configuration could introduce more complexity than expected.	High	Research the cloud setup early on before development. Extensive research also exposes potential skill gaps or unexpected complexities.
APIs or third-party services that are used may receive changes that require last-minute fixes.	Low	Choose established and actively maintained libraries with pinned versions.
The current version of programming languages may not have the proper libraries/modules we need during implementation.	Low	Find suitable replacements for the libraries and modules, or develop them ourselves if small enough to do so.
Losing access to files or data.	Low	Maintain a policy of all documents, files, and data being kept within the central repository.
The database may not be up to date with the current data.	Low	Implement automated data validation checks or synchronization scripts to ensure data consistency.
GitHub gets overwritten accidentally or reverts to an older version of the project.	Low	Implement branch protection rules to prevent forced pushes or accidental deletions on main branches, enforce a pull request review process for all code merges, and maintain regular repository backups
Builds of the project get lost before uploaded to GitHub	Low	Adopt a “commit early/commit often” policy for our source code. That way, if our build is ever lost, we can simply restart from the latest committed code, saving us a good amount of time.

4.3. External/Sponsor Risks

Risk	Likelihood	Strategies
------	------------	------------

Bad weather causes power or internet outages that might cause delays in the development process.	Low	Meet when possible in person and continue whatever development is possible during the outages.
Stakeholder feedback is received late, which may conflict with the development schedule.	High	Scheduled meetings with the sponsor, with direct communication (ie., email, cell) if necessary for information and feedback to be received incrementally.
The UI is not up to stakeholders' expectations, resulting in last-minute changes.	Medium	Schedule regular, iterative reviews with stakeholders to gather feedback early. Use wireframes aligned on UI/UX vision before development. Establish clear acceptance criteria for UI elements to prevent scope creep.

5. Software and Hardware Resource Requirements

For this project, we are going to use Java for creating the backend of the project and React for the front end of the project. All team members have experience with the Java language, and since it aligns with what is required for the project, we have chosen Java for the backend. Multiple team members have experience with React as a frontend framework, and React has many professionally developed libraries to ensure the UI is as clean as possible. Since this is cloud engineering, we will not have specific hardware for this project. Sabre Corp. is transitioning to Google systems during the semester, so Google's cloud systems will be the most likely choice at this time. We will use VS Code for the coding part of the lab and the development of React. Figma will be utilized to create wireframes/prototypes of the UI before implementation. The team will be able to establish the design and overall user experience before committing to code. The Figma prototypes will be used as a shared design for reference when developing React components.

6. Deliverables and Schedule

Task	Due Date
Project Management Plan	11:59:59 pm, September 11, 2026 (CDT)
Requirements Documentation	11:59:59 pm, September 25, 2026 (CDT)
Architecture Documentation	11:59:59 pm, October 16, 2026 (CDT)
Detailed Design Documentation	11:59:59 pm, November 6, 2026 (CDT)

Test Plan	11:59:59 pm, November 20, 2026 (CDT)
Final Project Presentation Slides and Demo	11:59:59 pm, December 1, 2026 (CDT)
Final Project Report	11:59:59 pm, December 4, 2026 (CDT)

The activities that we will be doing for this project will be many. The first will be doing a requirements analysis for this project to figure out what is needed for this project, seeing what risks can come from a product like this, and what should have our focus to complete first. This will help us understand what we are creating and will help us complete the Requirements Documentation assignment due on September 25th. This process would take us two weeks to complete. Everyone will be involved in this process to give input.

After requirements are finished, we will start doing research about cloud engineering and find other projects that are like ours to study the dos and don'ts of a project like this. This led us to develop the architecture of the project, letting us complete the Architecture Documentation Assignment due on October 16th. This process is estimated to take us three weeks to complete, and by then we will have figured out which groups will do the front and back end of the project as we start coding.

With the skeleton of the project completed and figured out, design comes next. This will help us flesh out the project more and get more feedback from the stakeholders of the project about how the UI should look and how the backend should run. This process will take us an estimated three weeks to be completed, and groups should stay the same. This will help us complete the Detailed Design Documentation assignment due on November 13th.

Then the next part is completing the project itself. With the architecture and design finalized, the next steps are to implement and test the software. This is estimated to take two weeks to complete, and everyone is going to come together to finish the documentation and work on testing. Alongside that, we will be making Final Presentation Slides for the whole class to see our project. The Test Plan is due on November 27th, and the Final Presentation Slides are due on the day of our presentation for this project.

With the code and testing finally complete, the final step of the project is the Final Project Report due on December 4th. This is important so everyone can work together to contribute to the report, and it should take an estimated week to complete, after which we should be done with the project.

7. Monitoring, Reporting, and Controlling Mechanisms

The team will monitor the project through weekly meetings, GitHub, Google Docs, and assigned tasks. Meetings will be held every Wednesday for team members to check the current status of the project, evaluate potential risks, implement necessary changes, discuss prior and ongoing tasks, review requirements and issues that may arise as progress is made, and prepare for upcoming deadlines. Tasks will also be distributed to each member fairly and effectively based on each member's responsibilities.

The team will also monitor project quality through feedback from our sponsor and professor. Project progress will also be reported during sponsor meetings. All meetings with our sponsor will be held every Friday to provide updates, ask questions about the requirements, receive feedback, and identify issues or any necessary changes. Discord will be used for team communication, and email for communication with the sponsor, professor, and TA.

All members are responsible for participating in weekly meetings. If an individual is not able to join the meeting, they have to check the announcement on Discord and follow up with the team's leader or other members. If delays, bugs, or other problems occur, the team will discuss them during team meetings or with sponsors to find solutions and adjust tasks or responsibilities as needed. If a team member is sick, unavailable, or drops off the class, their responsibilities will be distributed among the rest of our team to avoid delays.

GitHub is the primary tool that our group uses to manage the project. Our team will update documentation, code files, and other relevant files in the team's GitHub repository. Team members will commit and push their work to GitHub so changes can be tracked and everyone can access the latest version by using pull requests. Besides, Google Docs is another primary tool we will use to manage project documentation, planning, notes, and collaborative editing.

We chose these mechanisms because they are useful for maintaining version control, tracking requirements and progress, and ensuring that important information is properly stored, recoverable when needed, and available to all team members.

8. Professional Standards

All team members are expected to behave professionally towards each other, instructors, and sponsors. Team members are expected to join weekly meetings on time unless prior notice of absence or delay is provided. Transparent communication among team members is expected. Any issues that can cause delays in the outlined plans must be discussed in order to resolve them before the deadline. Tasks will be clearly defined and divided, and each team member is responsible for delivering by the due date. Scholastic dishonesty, including but not limited to plagiarism, unauthorized collaboration, and misrepresenting collaboration is prohibited. Please refer to Appendix A for guidelines on any occurrence of violation of these standards.

9. Configuration Management

- **Github Repository:** <https://github.com/kimchau123/SE-4385-Group-9>

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments
Alice	0.00	0.01	Document created	Shipped	Start of our project

Alice	0.01	0.02	Template added to Document	Shipped	
Alice	0.02	0.03	Lifecycle Model	Shipped	Waterfall is more appropriate for documentation and aligns with our deliverables
Jasmine	0.03	0.04	Project Abstract	Shipped	
Kim	0.04	0.05	Table of Contents	Shipped	
Tobenna	0.05	0.06	List of Tables	Shipped	
Tobenna/ Jasmine	0.06	0.07	Introduction	Shipped	
Alice	0.07	0.08	Project Organization	Shipped	
Team	0.08	0.09	Risk Analysis	Shipped	Aligned with the potential risks
Alice	0.09	0.10	Deliverables and Schedule	Shipped	Reviewed and understood
Kim	0.10	0.11	Monitoring, Reporting and Controlling Mechanisms	Shipped	
Jasmine	0.11	0.12	Professional Standards	Shipped	
Russel	0.12	0.13	Formatting Table of Contents	Shipped	
Russel	0.13	0.14	Risk Analysis Reformatting	Shipped	
Team	0.14	0.15	Configuration Management	Shipped	
Russel	0.15	0.16	Document Formatting	Shipped	
Russel	0.16	0.17	Schedule Due Dates	Shipped	
Team	0.17	0.18	Engineering Standards and Multiple Constraints	Shipped	

Team	0.18	0.19	Additional References	Shipped	References sought by team
Team	0.19	0.20	Appendix	Shipped	Team agrees
Russel	0.20	0.21	Added Figma to Software and Hardware Resources	Shipped	

10. Engineering Standards and Multiple Constraints

- [IEEE Std 1058-1998: Software Project Management Plans \[pdf\]](#)
- [PMBOK® Guide: Project Management Body of Knowledge \[pdf\]](#)
- [IEEE Std 12207: Software Life Cycle Processes \[pdf\]](#)
- [IEEE Std 15939: Measurement Process \[pdf\]](#)
- [ISO/IEC/IEEE Std 29148-2018: Systems and Software Engineering](#)
 - [Life Cycle Processes](#)
 - [Requirements Engineering \[pdf\]](#)

All group members and the sponsor, Sabre Corp., have agreed to use the above-listed and linked standards from PMBOK and IEEE. The team intends to use the IEEE 1058 and PMBOK Guide for standards involved with project planning and management, including how to approach the scope, scheduling, resources, risks, costs, and quality. The ISO/IEC/IEEE 12207 will provide our framework for our approach to our waterfall lifecycle through the phases of the project outlined in Section 6. ISO/IEC/IEEE 29148 will be used to ensure requirements meet expectations for definition, validity, verification, and traceability. ISO/IEC/IEEE 15939 will be used for measurement and analysis of relevant metrics for the project to monitor performance and inform our decision-making.

11. Additional References

- [Larson, E. and Gray, C., 2014. Project Management: The Managerial Process. McGraw Hill](#)
- [Humphrey, W.S. and Thomas, W.R., 2010. Reflections on Management: How to Manage Your Software Projects, Your Teams, Your Boss, and Yourself. Pearson Education](#)

Appendix A.

The Team and the sponsor have agreed to use the following Guidelines for unacceptable behavior.

Guideline:

On the first occurrence of unacceptable behavior, determine the circumstances involved, resolve the problem, and document the event in the meeting minutes.

On a second occurrence, notify the instructor of the problem. A meeting will be set up to evaluate the situation and resolve the problem.

On a third occurrence, again notify the instructor of the problem. A meeting will be set up to evaluate the situation and resolve the problem. At this point, the team will have the *option* of removing the team member. If removed, then the team member receives a pro-rated grade based on the number of weeks they have participated in the group.

Examples of unacceptable behavior may include not delivering on time, delivering poor quality work, missing team meetings, being unprepared for team meetings, disrespectful or rude behavior, etc. Reasons such as “too busy” or “I forgot”, or “my dog ate my design model” are unacceptable.

Valid reasons that must be considered include those listed for obtaining an incomplete standing in a course (illness, death in the family, travel for business or academic reasons, etc.)